

ME315A_Lab03

AUTHOR

João Vitor Expedito Silva de Oliveira RA: 294836

Tarefa

Considere novamente o problema de atrasos de vôos, disponível em <https://www.kaggle.com/usdot/flight-delays>. Nesta atividade, além dos dados de `flights.csv`, nós iremos utilizar informações disponíveis nos arquivos `airlines.csv` e `airports.csv`.

Resolução das atividades a serem desenvolvidas

1. Importe, utilizando o pacote `readr`, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados `flights`, `airlines` e `airports`.
 - Para o arquivo de aeroportos, importe apenas as colunas `IATA_CODE`, `CITY`, `STATE`, `LATITUDE`, `LONGITUDE`;
 - Para o arquivo de vôos:
 - a. Importe apenas as colunas `DESTINATION_AIRPORT` e `ARRIVAL_DELAY`;
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;
 - e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
 - f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
# Baixando pacotes
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

```
library(tidyr)

# Importação airlines
airlines <- read_csv('airlines.csv')
```

Rows: 14 Columns: 2

— Column specification —

Delimiter: ","

chr (2): IATA_CODE, AIRLINE

- ❗ Use `spec()` to retrieve the full column specification for this data.
- ❗ Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Importação airports
airports <- read_csv("airports.csv",
                     col_select = c(IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE)
                     )
```

Rows: 322 Columns: 5

— Column specification —

Delimiter: ","

chr (3): IATA_CODE, CITY, STATE

dbl (2): LATITUDE, LONGITUDE

- ❗ Use `spec()` to retrieve the full column specification for this data.
- ❗ Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Importação flights

## Função que processa cada chunk de 1 milhão de linhas
processa_pedaco <- function(pedaco, pos) {
  pedaco %>%
    select(DESTINATION_AIRPORT, ARRIVAL_DELAY) %>%      # item a
    filter(!startsWith(DESTINATION_AIRPORT, "1")) %>%   # item c
    drop_na() %>%                                         # item d
    group_by(DESTINATION_AIRPORT) %>%
    summarise(soma = sum(ARRIVAL_DELAY),                 # item e
              n = n(),
              .groups = "drop")
}

## Leitura em pedaços de 1 milhão de linhas (item b)
estatisticas_por_pedaco <- read_csv_chunked(
  "flights.csv",
  callback = DataFrameCallback$new(processa_pedaco),
  chunk_size = 1000000
)
```

— Column specification —

cols(

```
.default = col_double(),
AIRLINE = col_character(),
TAIL_NUMBER = col_character(),
ORIGIN_AIRPORT = col_character(),
DESTINATION_AIRPORT = col_character(),
SCHEDULED_DEPARTURE = col_character(),
DEPARTURE_TIME = col_character(),
WHEELS_OFF = col_character(),
WHEELS_ON = col_character(),
SCHEDULED_ARRIVAL = col_character(),
ARRIVAL_TIME = col_character(),
CANCELLATION_REASON = col_character()
)
```

i Use ``spec()`` for the full column specifications.

```
## Combinando as estatísticas de todos os pedaços (item f)
flights <- estatisticas_por_pedaco %>%
  group_by(DESTINATION_AIRPORT) %>%
  summarise(soma_total = sum(soma),
            n_total = sum(n)) %>%
  mutate(atraso_medio = soma_total / n_total)
```

2. Selecione a operação apropriada join para incluir, na tabela `flights`, as colunas `CITY` e `STATE` do objeto `airports`. Para executar esta tarefa:

a. Identifique a coluna que é a chave na tabela `flights`;

R: A coluna chave na tabela `flights` é `DESTINATION_AIRPORT`

b. Identifique a coluna que é a chave na tabela `airports`;

R: A coluna chave na tabela `airports` é `IATA_CODE`

c. Quais são os aeroportos que estão listados em `flights`, mas estão ausentes em `airports`?

```
flights %>%
  anti_join(
    airports,
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) %>%
  distinct(DESTINATION_AIRPORT)
```

A tibble: 0 × 1

i 1 variable: DESTINATION_AIRPORT <chr>

d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;

e. Armazene a tabela resultante no objeto `flights`.

```
flights <- left_join(
  flights,
  airports,
```

```
by = c("DESTINATION_AIRPORT" = "IATA_CODE")
)
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
airports %>%
  group_by(STATE) %>%
  summarise(
    quantidade_aeroportos = n()
  ) %>%
  arrange(desc(quantidade_aeroportos))
```

```
# A tibble: 54 × 2
  STATE quantidade_aeroportos
  <chr>          <int>
1 TX              24
2 CA              22
3 AK              19
4 FL              17
5 MI              15
6 NY              14
7 CO              10
8 MN               8
9 MT               8
10 NC              8
# i 44 more rows
```

4. Apresente um mapa representando todos os atrasos observados por aeroporto.

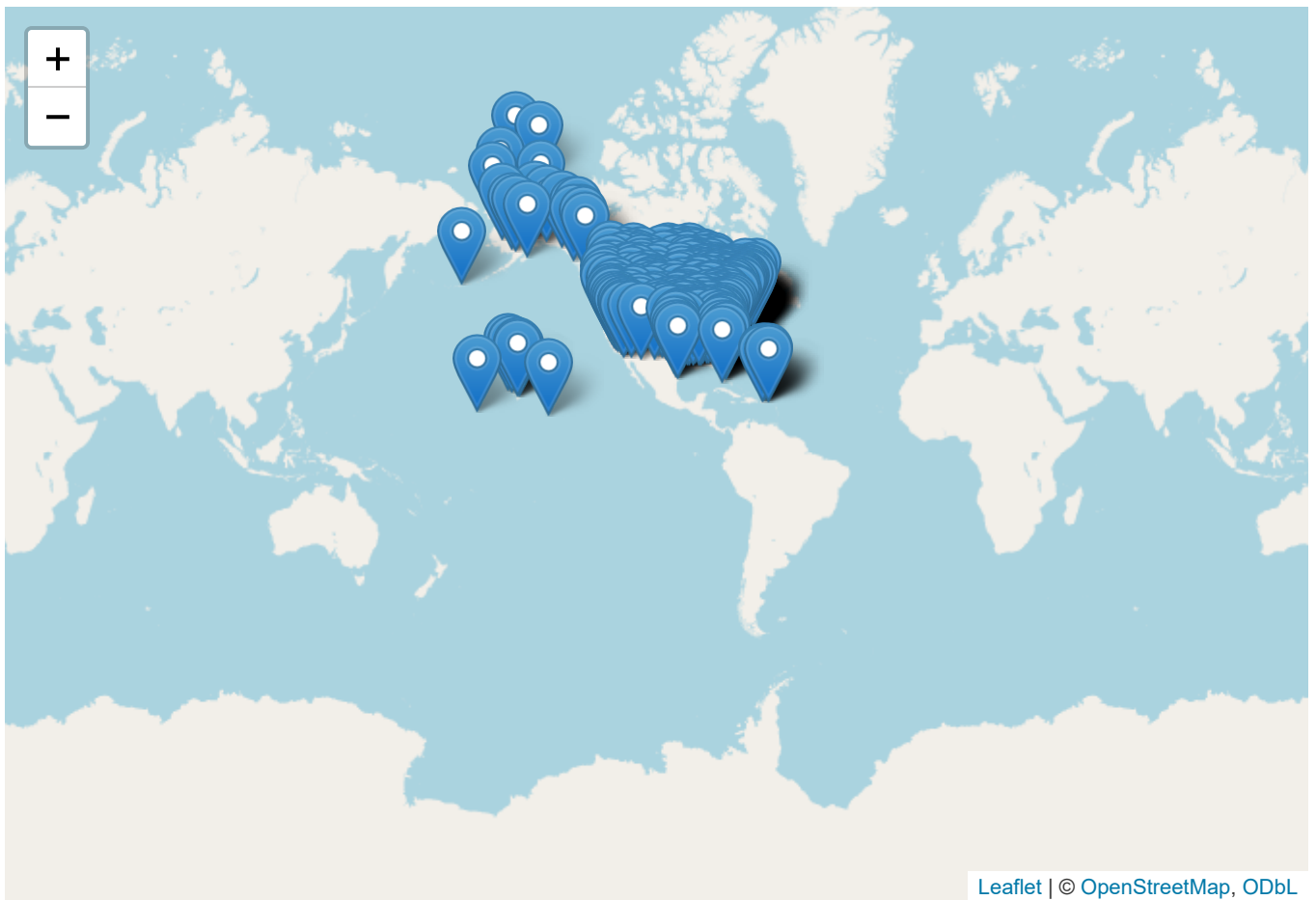
- Carregue o pacote `leaflet`;
- Combine os comandos `leaflet`, `addTiles` e `addMarkers` para a criação de um mapa básico;
- Armazene o grafico em b) numa variável chamada `this`;
- Adicione as duas linhas abaixo após a criação da variável `this` (apenas se você estiver usando Jupyter):

```
library(leaflet)

this <- leaflet(flights) %>%
  addTiles() %>%
  addMarkers(
    lng = ~LONGITUDE,
    lat = ~LATITUDE
  )
```

Warning in validateCoords(lng, lat, funcName): Data contains 3 rows with either missing or invalid lat/lon values and will be ignored

```
this
```



(OBS: Não consegui renderizar em pdf por conta desse último chunk apenas em html).

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 22:09:05 -03"
```